



Bóveda Digital

Secure Digital Document Vault — Entrega Final

Aguilar Pérez José Ramón · Carbajal Reyes Irvin Jair · Gómez Vázquez Juan Pablo · Sánchez Calvillo Saida Mayela

Criptografía Aplicada · Mayo 2026

¿Qué problema resuelve la Bóveda Digital?



El escenario: un laboratorio de investigación

Investigadores necesitan compartir documentos confidenciales (fórmulas, resultados experimentales) entre colegas. Hoy lo hacen por correo o en la nube, donde cualquier persona con acceso al servidor puede leer o modificar esos archivos sin que nadie se entere.

Sin la Bóveda, estos ataques son posibles



Espionaje

Un empleado del servidor puede leer todos los archivos en texto claro.



Manipulación

Alguien modifica los resultados del experimento y nadie lo detecta.



Suplantación

Un atacante sube un documento falso firmado con nombre ajeno.

Lo que garantiza la Bóveda Digital



Confidencialidad

El archivo solo puede leerlo el destinatario correcto.



Autenticidad

Puedes verificar exactamente quién envió el documento.



Integridad

Si alguien modifica el archivo, el sistema lo detecta y rechaza.

¿Cómo funciona? — Vista general del sistema



Remitente

Cifra el archivo y lo firma digitalmente antes de enviarlo. Todo sucede en su navegador.



Servidor

Almacena el archivo ya cifrado. Nunca puede leerlo ni conoce las llaves privadas.



Receptor

Descarga el archivo, verifica la firma y descifra el contenido con su llave privada.

El flujo completo, paso a paso:

1

Registrarse

Generar tus llaves públicas y privadas

2

Cifrar archivo

El archivo se bloquea solo para el receptor

3

Firmar

Se sella con tu firma digital personal

4

Subir

El archivo cifrado llega al servidor

5

Verificar

El receptor confirma quién lo envió

6

Descifrar

Solo el receptor puede abrirlo

El servidor almacena solo archivos ya cifrados — nunca tiene acceso al contenido original.

Proceso 1: Registro — ¿cómo se crean las llaves?

Analogía: piensa en un cajón con candado. Cuando te registras, el sistema te genera DOS cosas: un candado (llave pública, que le puedes dar a cualquiera) y una llave maestra (llave privada, solo tú la tienes). Para guardar mensajes, usas el candado ajeno. Para abrirlos, solo sirve la llave del dueño.



Se generan 2 pares de llaves

Llave de cifrado (X25519):
para que otros te envíen archivos cifrados.

Llave de firma (Ed25519):
para que firmes documentos con tu identidad.



Las llaves privadas se protegen

Tus llaves privadas se cifran con tu contraseña
(usando Argon2id, un algoritmo diseñado para que
las contraseñas sean muy difíciles de adivinar).
Se guardan en un archivo .encrypted en tu dispositivo.



Solo las llaves públicas van al servidor

El servidor guarda solo tu llave pública de cifrado
y tu llave pública de firma.
Nunca toca tus llaves privadas.
Esto significa que aunque hackeen el servidor,
no pueden descifrar tus archivos.



Descarga tu archivo de llaves

El archivo .encrypted se descarga una sola vez.
Es tu responsabilidad guardarlo en un lugar seguro.
Sin él y sin tu contraseña, no puedes acceder
a los archivos que te compartan.

Proceso 2: Cifrar y compartir un archivo

Cuando subes un archivo, TODO el proceso de cifrado ocurre en tu navegador antes de enviarlo al servidor. El servidor recibe el archivo ya bloqueado y no tiene ninguna forma de abrirlo.



Lo que hace especial a este proceso:

- ✓ El archivo original nunca sale de tu computadora sin cifrar.
- ✓ Cada destinatario recibe una versión distinta de la llave — si alguien es removido, la llave anterior ya no sirve.
- ✓ El sello (firma digital) garantiza que nadie modificó el archivo entre que lo enviaste y lo recibió el destinatario.
- ✓ El servidor solo almacena un blob de bytes que para él no tiene ningún significado.

Proceso 3: Recibir, verificar y abrir un archivo

Cuando un archivo te llega a la Bandeja, el sistema primero verifica que la firma sea auténtica ANTES de intentar abrirlo. Si alguien alteró el archivo o la firma, el sistema lo rechaza de inmediato sin exponer nada.



Descargar el paquete

Recibes el archivo cifrado, la firma del remitente y la llave del candado empaquetada para ti.



Verificar la firma digital (paso obligatorio)

El sistema comprueba: "¿esta firma coincide con la llave pública del supuesto remitente?"

Si no coincide → el archivo se rechaza completamente. No se abre nada.

FAIL CLOSED



Abrir la llave del candado

Cargas tu archivo de llaves (.encrypted) y escribes tu contraseña. Tu llave privada descifra la llave simétrica del archivo.



Descifrar el archivo

Con la llave simétrica se abre el archivo. El sistema verifica también que los metadatos (nombre, tamaño) no hayan sido cambiados.



El archivo original en tus manos

Descargas el documento original en tu navegador. Puedes ver quién lo envió, cuándo y que el contenido es íntegro.

Metadatos, estándares y cómo todo encaja

¿Qué son los metadatos y por qué importan?

Cada archivo cifrado lleva dentro información adicional que describe el archivo sin revelar su contenido. Estos metadatos también se protegen: si alguien los modifica, el sistema detecta la manipulación automáticamente.

Datos que viajan dentro del archivo cifrado:

Nombre del archivo	reporte_experimento_3.pdf
Algoritmo usado	ChaCha20-Poly1305
Fecha de creación	2026-05-25T14:30:00Z
Tamaño original	2,048,576 bytes
Versión del formato	1

Estándares y algoritmos utilizados

ChaCha20-Poly1305 (RFC 8439)

Cifrado + verificación de integridad en una sola operación. Recomendado por Google y Cloudflare para entornos sin aceleración de hardware.

Ed25519 (RFC 8032)

Firma digital moderna, muy rápida, sin vulnerabilidades conocidas. Cada firma es única e imposible de falsificar sin la llave privada.

X25519 (RFC 7748)

Intercambio de llaves. Permite que el remitente cifre para el receptor sin necesidad de que ambos estén conectados al mismo tiempo.

Argon2id (RFC 9106)

Protección de contraseñas. Diseñado para ser lento y consumir mucha memoria, haciendo que los ataques de fuerza bruta sean impracticables.

JSON Canónico (RFC 8785)

Garantiza que la misma información siempre produce los mismos bytes, necesario para que la firma sea verificable de forma determinista.

¿Cómo se desarrolló la aplicación?

Stack tecnológico



Frontend

React + Vite



Crypto

libsodium WASM



Backend

Spring Boot / Java 17



Base de datos

PostgreSQL



Testing

Vitest (firma, llaves, canon.)

Distribución de roles

Algoritmos

Aguilar Pérez J.R. — Diseño e implementación de todos los módulos criptográficos

Project Mgr

Carbajal Reyes I.J. — Coordinación del equipo, gestión del repositorio y entregas

Frontend

Gómez Vázquez J.P. — Interfaz de usuario, integración con la API y flujos UX

QA / Testing

Sánchez Calvillo S.M. — Pruebas unitarias, auditoría de seguridad y validación

Proceso de desarrollo — iteraciones por entregable

D2

AEAD + Nonce

Implementar ChaCha20-Poly1305 con metadatos canónicos como AAD

D3

Cifrado híbrido

X25519 para empaquetar la llave simétrica por cada receptor

D5

Firma digital

Ed25519 sobre el ciphertext completo + lista de destinatarios

D6

Protección de llaves

Argon2id KDF + formato BOV3 para el archivo de llaves

Audit

Auditoría

Identificar vulnerabilidades, aplicar fixes y documentar limitaciones

Auditoría de seguridad — lo que encontramos y lo que corregimos

Durante el desarrollo identificamos vulnerabilidades reales en nuestro propio código. Aquí explicamos qué eran, por qué importaban y qué hicimos.

RESUELTO

Contraseñas guardadas con un algoritmo de hashing rápido (BLAKE2b sin sal)

Problema: Un atacante con el archivo de llaves podía probar millones de contraseñas por segundo con una GPU.

Solución: Cambiamos a Argon2id con sal aleatoria. Ahora cada intento tarda ~1 segundo y requiere 64 MB de memoria.

RESUELTO

El orden de firma era incorrecto: se firmaba antes de cifrar

Problema: Un atacante que pudiera descifrar podía remover tu firma y poner la suya — suplantando tu identidad.

Solución: Ahora se cifra primero y después se firma el archivo ya cifrado. La firma cubre todo el paquete.

RESUELTO

El receptor podía abrir un archivo sin verificar la firma

Problema: Sin verificar la firma, no hay forma de saber si el archivo llegó íntegro o si fue modificado en el camino.

Solución: La verificación de firma es ahora el primer paso obligatorio. Si falla, el proceso se detiene completamente.

PENDIENTE

Las contraseñas viajan al servidor sin protección adicional

Problema: Aunque la conexión es HTTPS, las contraseñas se almacenan en texto plano en la base de datos del servidor.

Pendiente: Recomendación: aplicar BCrypt en el backend antes de guardar cualquier contraseña.

Conclusiones y trabajo futuro

Lo que logramos

Cifrado extremo a extremo — el servidor nunca accede al contenido de los documentos.

Autenticidad verificable — cualquier receptor puede confirmar quién firmó el documento.

Integridad garantizada — cualquier modificación al archivo es detectada y rechazada.

Llaves privadas protegidas — nunca salen del dispositivo del investigador.

Lo que aprendimos

En criptografía, el orden de las operaciones importa tanto como los algoritmos elegidos.

El eslabón más débil no es el cifrado — es la gestión de contraseñas y llaves.

Un sistema seguro no es el que nunca falla, sino el que falla de forma controlada.

Auditarse a uno mismo es tan valioso como construir el sistema.

Trabajo futuro — dentro del contexto del laboratorio

Soporte para múltiples firmantes, para experimentos que requieren validación de varios investigadores.

Revocación de acceso — que el dueño del documento pueda remover a un receptor después de haberlo compartido.

Registro de auditoría — un historial de quién abrió cada documento y cuándo, sin comprometer la privacidad.